Week5

Be aware: This document does not cover all possible correct answers. If your answer differs, it does not necessarily mean it is incorrect.

Q1) * 10 point

```
class Counter;
    int count = 0;
    static int total = 0;

    function new();
        count++;
        total++;
    endfunction

    function void display();
        $display("Count: %0d, Total: %0d", count, total);
    endfunction
endclass

module test;
    initial begin
        Counter c1 = new();
        Counter c2 = new();
        Counter c3 = new();

        c1.display();
        c2.display();
        c3.display();
    end
endmodule
```

```
(A)
Count: 1, Total: 3
Count: 1, Total: 3
Count: 1, Total: 3
```

○ A

```
(B)
Count: 1, Total: 1
Count: 1, Total: 2
Count: 1, Total: 3
```

○ B

```
(C)
Count: 1, Total: 3
Count: 2, Total: 3
Count: 3, Total: 3
```

```
(D)
Count: 3, Total: 1
Count: 3, Total: 1
Count: 3, Total: 1
```

kareem Waseem

Answer: **A**

Explanation: **"Total" is a static variable and hence is a shared variable among all the objects and gets updated from any of them. Here, it reflected the number of objects created. Review static variables in the notebook.**

Q2 *

Which UVM function triggers the phases execution?

- ◯ uvm_top()
- ◯ run_test()
- ◯ start_of_simulation_phase()
- ◯ uvm_component_utils()

Answer: **run_test()**

kareem Waseem

Q3) *

10 points

In UVM, regarding build_phase and connect_phase, run_phase, which of the following statements are <u>TRUE</u>?

(Note: parent means the higher component in hierarchy while child is lower component e.g. Environment is the parent corresponding to Driver)

(Select all correct answers)

- [ ] In build_phase, parent components create their child components.
- [ ] In build_phase, children are built before the parent.
- [ ] In connect_phase, children connect their ports/exports first.
- [ ] In connect_phase, parent connects first.
- [ ] In run_phase, children must run before parent
- [ ] In run_phase, parents must run before children

<u>Answer:</u>

- **In build phase, parent components create their child components. (first choice)**
- **In connect_phase, children connect their ports/exports first. (third choice)**

kareem Waseem

Explanation:

- **Build_phase**
  - **In build_phase, parent components create their child components.**
  - **build_phase is top-down.**
  - **Example: env.build_phase creates agent, then inside agent.build_phase it creates driver/monitor.**
  - **Parents must exist before a child can be constructed.**
- **Connect_phase**
  - **In connect_phase, children connect their ports/exports first.**
  - **connect_phase is bottom-up.**
  - **Execution order: driver.connect_phase → agent.connect_phase → env.connect_phase.**
  - **This allows children's ports/exports to exist and be ready when the parent connects higher-level connections.**
- **Run_phase**
  - **run_phase is a parallel task phase.**
  - **Child and parent run concurrently; there is no strict before/after order.**

Q4) * 10 points

What does "env.drv" specify?

```
uvm_config_db#(virtual my_if)::set(this, "env.drv", "vif", my_if_h);
```

- ◯ The type of the configuration variable.
- ◯ The name of the field to be retrieved later.
- ◯ The handle of the virtual interface being stored.
- ◯ The component that will have visibility of this config entry.

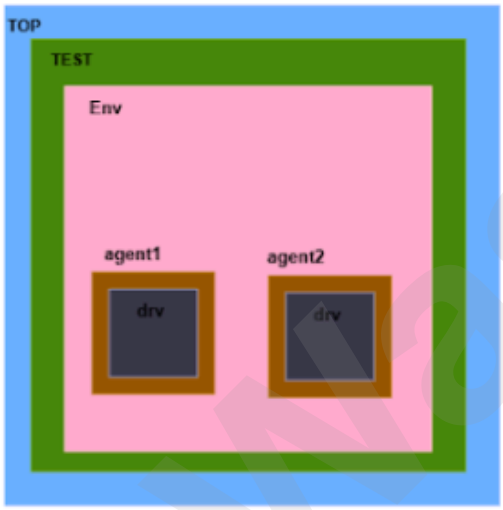
Answer: **The component that will have visibility of this config entry.**

kareem Waseem

Q5) Which components can successfully retrieve this interface? * 10 points

In the previous question, if we change the line to set the virtual interface inside my test to be:

```
uvm_config_db#(virtual my_if)::set(this, "env.*", "vif", vif_h);
```



Refer to this image to understand the agents' place in the hierarchy ***(agent is a component class, same as driver class)***

- ◯ Only env
- ◯ Only env.agent1 and env.agent2
- ◯ Only env.agent1, env.agent2, env.agent1.drv and env.agent2.drv
- ◯ All components under my_test

<u>Answer:</u> Only env.agent1, env.agent2, env.agent1.drv and env.agent2.drv

kareem waseem

Q6)

Given that get_name() and get_full_name() are functions used with UVM components and objects :

get_name() → returns only the local name of the object/component (name written inside create line).

get_full_name() → returns the full hierarchical path (e.g. uvm_test_top.env.drv)

What will be the expected output from this code?

```
// Config object (uvm_object)
class cfg extends uvm_object;
  `uvm_object_utils(cfg)

  function new(string name = "cfg");
    super.new(name);
  endfunction
endclass

// Driver component
class my_driver extends uvm_driver #(uvm_sequence_item);
  `uvm_component_utils(my_driver)

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
endclass


// Env component containing driver
class my_env extends uvm_env;
  `uvm_component_utils(my_env)
  my_driver drv;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    drv = my_driver::type_id::create("drv", this);
  endfunction
endclass
```

kareem Waseem

```
// Test component containing env
class my_test extends uvm_test;
  `uvm_component_utils(my_test)
  my_env env;
  cfg c;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = my_env::type_id::create("env", this);

    c = cfg::type_id::create("cfg_obj");
  endfunction

  task run_phase(uvm_phase phase);
    $display("Driver get_name()       = %s", env.drv.get_name());
    $display("Driver get_full_name() = %s", env.drv.get_full_name());
    $display("Cfg get_name()          = %s", c.get_name());
    $display("Cfg get_full_name()     = %s", c.get_full_name());
  endtask
endclass


module tb;
  initial begin
    run_test("my_test");
  end
endmodule
```

```
# Driver get_name()       = drv
# Driver get_full_name() = uvm_test_top.env.drv
# Cfg get_name()          = cfg_obj
# Cfg get_full_name()     = uvm_test_top.env.cfg_obj
```

A

```
# Driver get_name()       = drv
# Driver get_full_name() = uvm_test_top.env.drv
# Cfg get_name()          = cfg_obj
# Cfg get_full_name()     = uvm_test_top.cfg_obj
```

B

```
# Driver get_name()       = drv
# Driver get_full_name() = uvm_test_top.env.drv
# Cfg get_name()          = cfg_obj
# Cfg get_full_name()     = uvm_test_top.env.drv.cfg_obj
```

C

```
# Driver get_name()       = drv
# Driver get_full_name() = uvm_test_top.env.drv
# Cfg get_name()          = cfg_obj
# Cfg get_full_name()     = cfg_obj
```

D

kareem Waseem

Answer: **D**

> **Note that Cfg is an object not a component and for a uvm_object, get_name() and get_full_name() return the same value because it has no hierarchy.**

Q7) In a UVM, what could cause the simulation to hang and never end. *

- ◯ The environment did not call uvm_config_db::set() to pass the interface handle.
- ◯ The run_phase raised an objection but never dropped it.
- ◯ The UVM factory was not used to create components.
- ◯ The DUT finished execution too early.

Answer: **The run_phase raised an objection but never dropped it.**

Q8) *

What is the purpose of the UVM factory?

- ◯ To start and stop simulations
- ◯ To clone DUT components
- ◯ To enable dynamic object creation and override
- ◯ To randomize signal values

Answer: **To enable dynamic object creation and override**

Kareem Waseem

Q9) *

10 p

Which phase is commonly used for test stimulus generation in UVM?

- ◯ build_phase
- ◯ connect_phase
- ◯ run_phase
- ◯ report_phase

<u>Answer:</u> run_phase (it is the only phase that can consume time)

kareem Waseem

Q10) *

According to the following snippets, the code will

```
function void build_phase(uvm_phase phase);
   super.build_phase(phase);
      if(!uvm_config_db #(q10_config)::get(this , "" , "CFG" , q10_cfg))begin
        `uvm_fatal("build_phase","unable to get configuration object in the driver")
      end
endfunction


function void connect_phase(uvm_phase phase);
   super.connect_phase(phase);
endfunction

task run_phase(uvm_phase phase);
   super.run_phase(phase);
      forever begin
         q10_vif_driver.rst_n =$random;
         q10_vif_driver.received_data = $random;
         @(negedge q10_vif_driver.clk);
      end
endtask
```

- ◯ Compile successfully and the design values will be randomized successfully
- ◯ Compile successfully and the design values will not be randomized successfully
- ◯ Will not compile successfully

<u>Answer:</u> Compile successfully, and the design values will not be randomized successfully

<u>Explanation:</u>

This is because of the missing connection in the connect phase between configuration object q10_cfg and q10_vif_driver. What is happening here is that q10_vif_driver is being driven which is fine but how can these values get to the DUT without connecting it to the virtual interface inside the config object??

kareem Waseem